Book API Technical Report

1. Project overview

This project is a database-driven web API for managing and exploring book metadata. It was developed for the XJCO3011 Web Services and Web Data coursework using Django and Django REST Framework. The final system supports full CRUD operations, public dataset import, query filtering, rating-ranked search, weighted book recommendations, collection statistics, authentication for write operations, and interactive API documentation.The idea of this project is a book metadata and recommendation API based on a public dataset.

2. Technology stack and rationale

Programming language

Python was chosen because it allows fast development, has strong ecosystem support for web APIs and data handling, and fits naturally with the technologies introduced in the module.

Frameworks and libraries

- Django provides the project structure, ORM, admin panel, and migration system.
- Django REST Framework provides serializers, viewsets, request parsing, validation, and the browsable API.
- drf-spectacular provides OpenAPI schema generation together with Swagger UI documentation.

This stack was selected because it offers a good balance between speed of development, maintainability, and clarity for coursework presentation.

Database

SQLite was used as the SQL database. It is lightweight, easy to run locally, and fully suitable for the coursework requirement of a relational database-backed API.

3. Public dataset integration

The project integrates a public CSV dataset stored at archive/books.csv. The dataset used for this coursework is from *Goodreads-books* (Soumik, 2020).

According to the dataset description, it provides Goodreads-style book metadata. The current imported CSV contains book metadata fields including: title, authors, average rating, ISBN and ISBN13, language code, number of pages, ratings count, text reviews count, publication date, publisher, book ID.

The Django Book model stores the fields that are most useful for the API demonstration. Since the new dataset does not include genre, description, or thumbnail values, imported records use Uncategorised as the genre and leave description and thumbnail blank unless edited manually through the API or admin interface.

To support this dataset, a custom Django management command was implemented:

```
python manage.py import_books_dataset --replace
```

This command imports the dataset into the SQLite database and can optionally replace existing records. Using a public dataset makes the project better aligned with the coursework brief and makes the API more realistic to demonstrate.

4. Data model

The application uses one main model: Book. Each book stores: bookid, title, author, genre, published_year, description, pages, publisher, language, average_rating, ratings_count, thumbnail

The id field remains the normal Django database primary key, while bookid is a separate sequential coursework-facing identifier. After dataset import, bookid is renumbered from 1 to the number of stored records so that books can be looked up using a cleaner demonstration ID without changing the database primary key. The model is still simple enough to explain clearly, but it is now rich enough to support metadata browsing, filtering, rating-ranked search, and weighted similarity-based recommendation behaviour based on the imported public dataset.

5. API design

The final API endpoints are:

- GET /api/books/
- POST /api/books/
- GET /api/books/{id}/
- GET /api/books/by-bookid/{bookid}/
- PUT /api/books/{id}/
- DELETE /api/books/{id}/
- GET /api/books/stats/
- GET /api/books/search/
- GET /api/books/recommendations/similar/

This API design keeps the core CRUD structure compact while adding functionality that better fits the book dataset use case. Genre-based lookup is still available through filtering on the main collection endpoint for manually categorised records, while rating-ranked search is handled through a dedicated search endpoint and similarity-based recommendations are handled through a separate weighted recommendation endpoint.

6. Search logic

The search endpoint is designed for user-driven retrieval from the current Goodreads CSV dataset. It allows results to be narrowed by title, author, publisher, language, and minimum rating, then ranks matching books by `average_rating` first and uses `ratings_count` as a secondary signal when ratings are tied. The endpoint returns up to the requested limit, so higher-rated and better-supported matches appear before weaker ones.

7. Recommendation logic

The project also includes a separate similarity-based recommendation endpoint so that recommendation is not confused with ordinary search. The endpoint accepts a seed book identified by bookid, then scores other books against that seed.

The weighted scoring formula is:

```
base_score =
  0.40 * title_score + 0.15 * author_score + 0.10 * publisher_score + 0.15 *
  language_score + 0.15 * rating_score + 0.05 * year_score
```

Where:

- title_score combines title keyword overlap and title sequence similarity,
- author_score rewards shared author information but does not dominate the final result,
- publisher_score rewards matching publishers,
- language_score rewards books in the same language,
- rating_score combines average rating with rating-count confidence,
- year_score rewards books published in a similar period.

To improve result diversity, the final list is not produced by raw score alone. Instead, the system applies:

- ***author repetition penalties,***
- ***title duplication penalties for very similar titles.***

This means the system avoids filling the top results with too many books by the same author or too many near-duplicate title variants. This produces a recommendation list that is more balanced and more useful than a simple same-author lookup.

8. Authentication and permissions

The API uses Basic Authentication for write operations. Public GET requests can be used without logging in, while POST, PUT, and DELETE require authentication.
This design was chosen because:

- it is easy to demonstrate in Swagger,
- it keeps read-only exploration simple,
- it protects database-modifying operations,
- it matches the coursework requirement to document authentication where applicable.

9. Validation and error handling

Validation is handled in the serializer layer to ensure that required fields are not blank and that years and ratings stay within valid ranges. The API returns standard REST status codes, including `200`, `201`, and `204` for successful operations, and `400`, `401`, and `404` for validation errors, unauthorised write attempts, and missing resources.

10. Documentation approach

The project includes multiple forms of documentation: *README.md* for setup instructions and project overview, *api_docs.md* for manual API documentation, generated OpenAPI schema in *schema.yml*, *Swagger UI* for live interactive documentation, a PDF export of the API documentation for submission.

11. Testing approach

Automated API tests were implemented using Django REST Framework test tools to verify the main system behaviour, including CRUD operations, filtering, search, statistics, recommendations, validation, authentication, and Swagger UI availability. These tests were used throughout development to confirm that the API remained stable after design changes.

12. Challenges and Limitations

Several design decisions shaped the final system. The project was developed from a basic CRUD API into a public-dataset book metadata API to better match the coursework brief, and search and recommendation were separated so that they served distinct purposes more clearly.

The project still has some limitations: it uses a single main model, is intended mainly for local coursework demonstration, relies on basic authentication, does not yet support pagination, and uses manually chosen recommendation weights. In addition, search and recommendation quality are constrained by the completeness of the imported dataset.

13. Future improvements

Possible future improvements include:

- adding related models such as reviews, users, reading lists, or favourites,
- tuning weights using evaluation experiments or user feedback,
- expanding recommendation features with richer metadata or user interaction history,
- introducing pagination for larger result sets,
- deploying the project to a hosted platform,
- replacing Basic Authentication with token-based or JWT-based authentication,
- importing and merging additional public datasets for richer metadata.

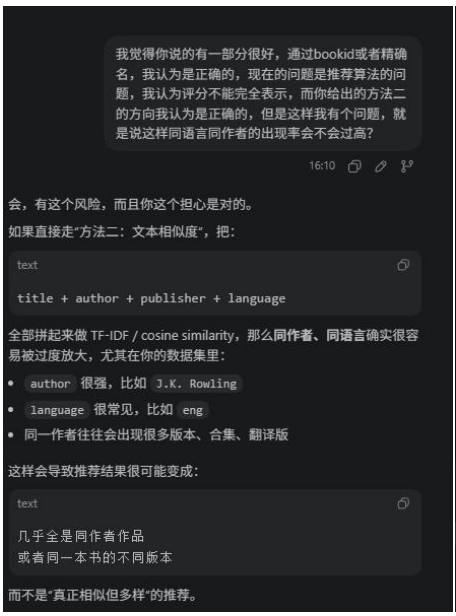
14. Generative AI declaration

Generative AI was used as a development support tool during this project. Its uses included:

- planning the API structure,
- refining endpoint design,
- processing and importing the dataset as shown in Figure 1

Figure1. Importing the dataset with AI

- processing and importing the dataset as shown in Figure 2



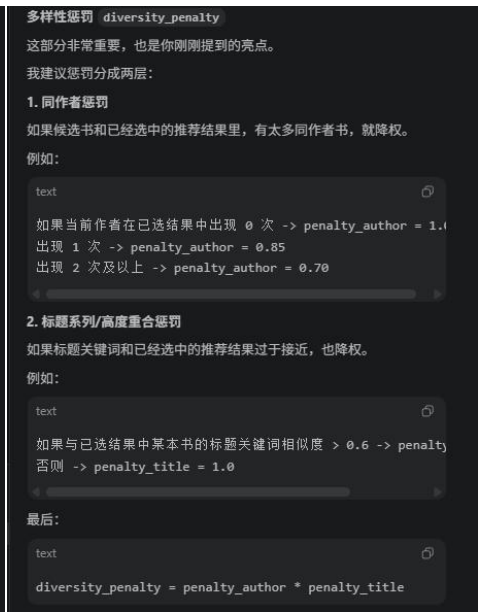


Figure2(a) Optimizing the book recommendation system with AI

Figure2(b) Optimizing the book recommendation system with AI

In this coursework, the use of AI was mainly focused on dataset processing, recommendation algorithm improvement, and code quality refinement. Since the project involved two changes of dataset during development, AI assisted in writing the dataset import script and helped preprocess irregular data so that it could be imported and read more effectively by the system.

In addition, the optimisation of the recommendation algorithm was partly inspired by AI suggestions, particularly the use of title text similarity and diversity penalties. These ideas helped me develop a more effective book recommendation algorithm and significantly reduced repetitive recommendation results.

Finally, AI also supported improvements to code style and consistency, including indentation, formatting, and general code organisation. This helped keep the project structure cleaner and made the interface and implementation easier to maintain.

15. References

Soumik. 2020. *Goodreads-books*. [Online]. [Accessed 20 April 2026]. Available from: https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks

Appendix

Deployment links

The deployed application is available at:

- https://whslqy.pythonanywhere.com/
- https://whslqy.pythonanywhere.com/api/docs/swagger/

Admin id:admin

Admin password: Admin12345!

Github links：

https://github.com/whslqy/web_cwk1 （containing code and API documentation）